BUITEMS

NETWORK TRAFFIC ANALYZER USING PYTHON

Project Report

Submitted By

FAHAD AHMED (69192)

MUNEEB UR REHMAN (69247)

BAHADUR KHAN (69572)

Contents

1. Introduction

Network traffic analysis is an essential part of modern computer networking and cybersecurity. Every device connected to a network continuously exchanges data through packets. Monitoring these packets helps administrators understand network behavior, troubleshoot connectivity issues, detect abnormal traffic patterns, and improve network performance.

The **Network Traffic Analyzer Using Python** is a command-line application developed to capture live network packets, store packet information, perform statistical analysis, and generate graphical reports. The application uses Python libraries such as **Scapy**, **Pandas**, and **Matplotlib** to provide a simple yet effective network monitoring solution.

Instead of inspecting packet payloads, the project focuses on packet metadata, including source IP address, destination IP address, communication protocol, source port, destination port, packet size, and timestamp. The collected information is stored in CSV format for further analysis and visualization.

The project demonstrates the practical application of Python programming, network packet analysis, data processing, and data visualization in a modular and easy-to-understand implementation.

---

2. Problem Statement

Monitoring network activity manually is difficult because thousands of packets are transmitted every second across a network. Traditional packet analysis tools provide extensive functionality but are often complex for beginners and educational purposes.

The objective of this project is to develop a lightweight Python-based network traffic analyzer capable of:

- Capturing live network packets.
- Extracting essential packet information.
- Storing captured packets in a structured CSV file.

- Performing statistical analysis on captured traffic.
- Generating graphical representations of network activity.

The system provides a simplified solution suitable for learning network traffic analysis and understanding communication patterns.

---

3. Project Objectives

The main objectives of this project are:

- Capture live network packets in real time.
- Identify the protocol associated with each packet.
- Record source and destination IP addresses.
- Record source and destination port numbers.
- Calculate packet size.
- Store packet information in CSV format.
- Analyze captured traffic statistically.
- Generate graphical reports for better visualization.
- Develop a modular and reusable Python application.

---

4. System Requirements

Hardware Requirements

- Intel Core i3 Processor or higher
- 4 GB RAM or above
- 100 MB Available Storage
- Internet Connection

Software Requirements

- Windows 10 / Windows 11
- Python 3.10 or higher
- Visual Studio Code
- Command Prompt / PowerShell

Python Libraries

- Scapy
- Pandas
- Matplotlib
- CSV
- Datetime
- OS

---

5. Technologies Used

Technology	**Purpose**
Python	Core Programming Language
Scapy	Packet Capturing
Pandas	Data Processing and Analysis
Matplotlib	Graph Generation
CSV	Data Storage
Datetime	Timestamp Generation

OS Module	File Management
Wireshark	Packet Inspection and Traffic Verification

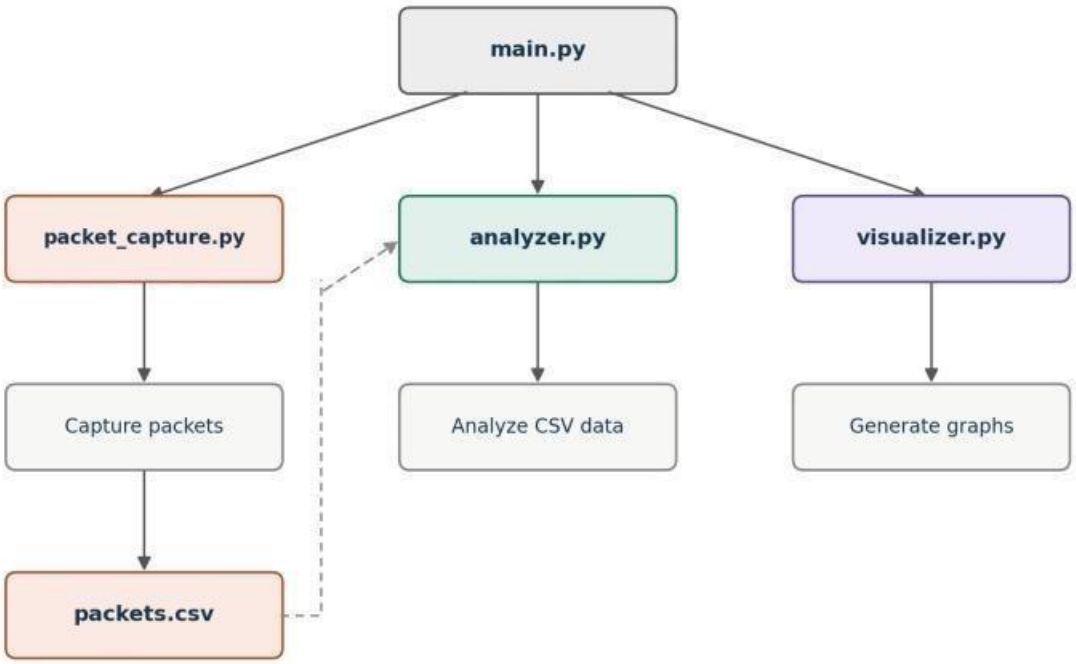
Visual Studio Code Development Environment

---

6. Project Architecture

The project follows a modular architecture where each component performs a specific responsibility.

The diagram below shows how main.py orchestrates packet capture, analysis, and visualization.



Flow summary

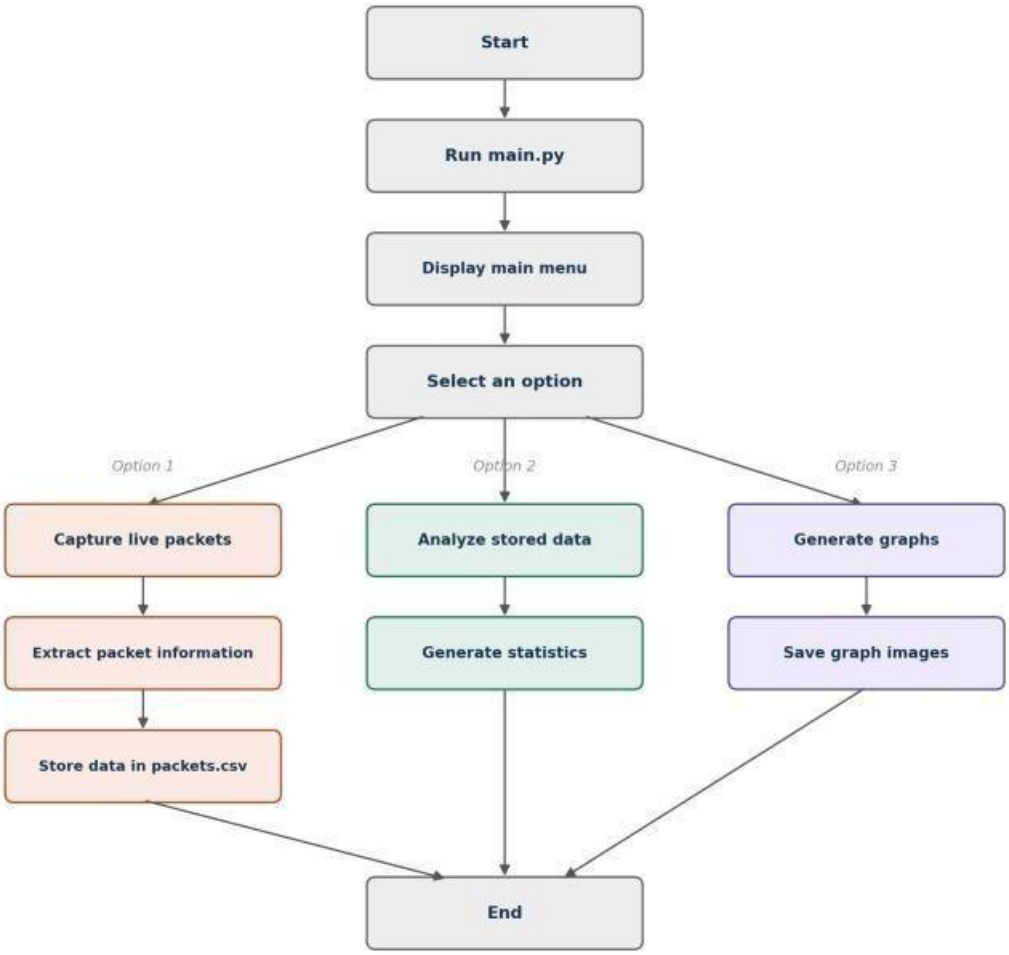
1. main.py starts the pipeline and calls each module in turn.
2. packet_capture.py captures live packets and writes them to packets.csv.
3. analyzer.py reads packets.csv and analyzes the captured data.
4. visualizer.py takes the analyzed output and generates graphs.

---

7. Project Workflow

The application follows the sequence below:

The diagram below shows the main menu flow and the three available options.



Flow summary

1. The program starts and runs main.py, which displays the main menu.
2. The user selects one of three options.
3. Option 1 captures live packets, extracts packet information, and stores it in packets.csv.
4. Option 2 analyzes the stored data and generates statistics.
5. Option 3 generates graphs and saves them as images.
6. Each path leads to the end of the program.

8. Module Description

8.1 Main Module (main.py)

The main module acts as the entry point of the application. It presents a menu-driven interface that allows the user to select one of three operations:

- Capture Live Packets
- Analyze Saved Packets
- Generate Graphs
- Exit Application

It coordinates the execution of all other modules.

8.2 Packet Capture Module (packet_capture.py)

This module captures live packets using the Scapy library.

For each captured packet, it extracts:

- Timestamp
- Source IP
- Destination IP
- Protocol
- Source Port
- Destination Port
- Packet Size

The extracted information is displayed on the terminal and appended to the CSV file for permanent storage.

8.3 Analysis Module (analyzer.py)

The analyzer module reads the stored CSV file using Pandas and computes statistical summaries of the captured traffic.

The generated statistics include:

- Total packets captured
- Protocol distribution
- Top source IP addresses
- Top destination IP addresses
- Top destination ports
- Average packet size

These statistics help users understand the characteristics of network traffic.

8.4 Visualization Module (visualizer.py)

This module converts the analyzed packet data into graphical reports using Matplotlib.

The generated visualizations include:

- Protocol Distribution (Bar Chart)
- Protocol Distribution (Pie Chart)
- Top 5 Source IP Addresses
- Top 5 Destination Ports
- Packet Size Distribution Histogram

All graphs are automatically stored in the **graphs** folder.

---

9. Project Implementation

The implementation consists of the following steps:

Step 1

Install the required Python libraries. pip install

scapy pandas matplotlib

Step 2

Run the application. python

main.py

Step 3

Select **Capture Live Packets** to monitor network traffic.

Step 4

Captured packets are automatically stored in: data/packets.csv

Step 5

Run the **Analyze Saved Packets** option to generate traffic statistics.

Step 6

Run the **Generate Graphs** option to create graphical reports.

The graphs are stored in:

graphs/

9.1 Packet Verification Using Wireshark

Wireshark was used during the testing phase to validate the packets captured by the Pythonbased Network Traffic Analyzer. It was used to inspect TCP, UDP, and DNS traffic and verify that the captured packet information matched the application's output. Wireshark served as a verification tool only, while packet capturing, analysis, CSV storage, and graph generation were performed using Python.

---

10. Results and Output

The application successfully captured live network packets and extracted important metadata including source IP, destination IP, protocol type, port numbers, packet size, and timestamp.

The captured packets were stored successfully in CSV format and later analyzed using the Pandas library.

The analysis generated useful statistics such as protocol distribution, frequently communicating IP addresses, destination ports, and average packet size.

The visualization module generated multiple graphical reports, allowing users to interpret network activity more effectively through charts.

The generated outputs include:

- packets.csv
- Protocol Distribution Bar Chart
- Protocol Distribution Pie Chart
- Top Source IP Graph
- Top Destination Port Graph
- Packet Size Distribution Graph

---

11. Advantages

- Real-time packet capturing.
- Modular project structure.
- Automatic CSV storage.
- Statistical traffic analysis.
- Graphical visualization of results.
- Beginner-friendly implementation.
- Lightweight and efficient.
- Easy to extend with additional features.

---

12. Limitations

- Requires administrator privileges for packet capture.
- Captures traffic only from the selected network interface.

- Does not inspect encrypted payload content.
- Command-line interface only.
- No advanced packet filtering.

---

13. Future Enhancements

The project can be extended by adding:

- Graphical User Interface (GUI)
- Real-time dashboard
- Packet filtering based on protocol or IP
- Export reports to PDF or Excel
- IP geolocation
- Network intrusion detection
- Database integration
- Live monitoring dashboard
- Alert system for suspicious traffic
- Machine learning-based anomaly detection

---

14. Conclusion

The **Network Traffic Analyzer Using Python** successfully demonstrates the process of capturing, analyzing, storing, and visualizing network traffic in real time. The project integrates packet sniffing, structured data storage, statistical analysis, and graphical visualization into a single modular application.

The use of Scapy enables efficient packet capture, while Pandas simplifies data processing and Matplotlib provides meaningful graphical insights into captured traffic. The modular

architecture makes the project easy to understand, maintain, and extend with advanced networking or cybersecurity features.

Overall, the project provides a practical learning experience in Python programming, computer networking, and data analysis while serving as a solid foundation for future network monitoring applications.

---

15. References

1. Python Software Foundation. *Python Documentation*. https://docs.python.org/3/
2. Scapy Documentation. https://scapy.readthedocs.io/
3. Pandas Documentation. https://pandas.pydata.org/docs/ 4.

   Matplotlib Documentation. https://matplotlib.org/stable/
5. RFC 791 – Internet Protocol (IP).
6. RFC 793 – Transmission Control Protocol (TCP).
7. RFC 768 – User Datagram Protocol (UDP).